



CS3244 Machine Learning

FINAL REPORT

Exploring Pairs Trading Strategy to Identify Profitable Stock Relationships

Group 23

Group Leader: Priscilla Ashley Wijaya

Mentor: Zhao Junchuan

Group Members

Catherine Anne Listijo	(A0305671H)
David Michael Indraputra	(A0276057X)
Felix Tanwira	(A0305741L)
Kenneth Christopher Hendra	(A0303278E)
Mabel Augustine Anggoro	(A0284680W)
Priscilla Ashley Wijaya	(A0305657Y)

April 19, 2026

Contents

1	Introduction	1
2	Dataset	1
3	Methodology	1
3.1	Data Preprocessing and Pipeline Setup	1
3.2	Exploratory Data Analysis	2
3.3	Pair Discovery and Statistical Testing	2
3.4	Feature Engineering and Dataset Construction	3
3.5	Spread Modelling and Signal Prediction	4
3.6	Signal Standardization and Decision Logic	6
3.7	Trading Strategy and Backtesting	6
3.8	Robustness and Final Evaluation	7
4	Results and Discussion	7
4.1	Model Comparison	7
4.2	Sensitivity Analysis	8
4.3	SHAP Analysis	8
4.4	Microscopic Evaluation	9
5	Conclusions	10
5.1	Limitations	10
5.2	Future Directions	10

1 Introduction

Pairs trading is a market-neutral statistical arbitrage strategy that profits from the relative price movement between two historically linked assets rather than from broader market direction. When the spread between a pair widens beyond its historical equilibrium, a trader shorts the over-performing asset and buys the under-performing one, earning a return when the spread reverts.

The foundational distance-based method of Gatev et al. [?] and the cointegration framework of Vidya-murthy [4] established pairs trading as rigorous, but both rely on static selection rules and fixed $\pm 2\sigma$ entry thresholds that struggle when hedge ratios drift or market regimes shift. Machine learning is a natural fit for these limitations: pair discovery scales as $O(N^2)$, so a 1,000-stock universe generates roughly 500,000 mostly-spurious candidates that make brute-force cointegration testing statistically unreliable and computationally wasteful, and supervised models can learn to distinguish temporary divergences from structural breaks that a fixed threshold treats identically.

Our project extends the Sarmiento and Horta [3] pipeline with four contributions: (i) a leakage-free walk-forward framework combining PCA, OPTICS clustering, and a four-criterion cointegration filter; (ii) a Kalman-filter hedge ratio paired with a β -locked labelling scheme that isolates spread prediction from hedge-ratio drift; (iii) a comparative evaluation of six models (OU, ARMA, Linear Regression, XGBoost, LSTM, Encoder-Decoder) chosen to answer distinct hypotheses about where predictive signal lives, under a unified Directional Weighted MSE loss and a shared execution layer; and (iv) a microscopic error analysis linking hedge-ratio stability to trade outcomes. Together, these address the central question of the project: ***Given a universe of liquid US equities with daily OHLCV data, can a machine learning pipeline that jointly learns which pairs are structurally cointegrated and when their spreads will mean-revert deliver risk-adjusted returns superior to static threshold-based pairs trading, once realistic transaction costs and non-stationary market regimes are accounted for?***

2 Dataset

We utilized the Huge Stock Market Dataset [2], sourced from Kaggle, which contains historical daily price and volume data for approximately 8,400 US-based stocks and ETFs. Each record includes Date, Open, High, Low, Close, Volume, and Open Interest (OpenInt).

3 Methodology

3.1 Data Preprocessing and Pipeline Setup

Given the computational complexity of pairwise comparisons $O(N^2)$, it is necessary to prune the initial dataset of 8,400 companies or assets. Based on the average daily dollar volume, calculated as the mean of each stock’s closing price multiplied by its trading volume, we filtered to the top 1,000 most liquid stocks to ensure that the trading strategy remains practically viable and resistant to the skewness of illiquid assets. To maintain strict experimental integrity and prevent look-ahead bias, we implemented a fixed sliding window backtesting framework of size 3 years following Sarmiento (2021) [3], which balances having enough data (756 observations) for statistically significant cointegration tests against staying short enough that the pair relationships remain structurally valid within the window.

The data is sampled at a daily granularity, providing roughly 252 data points per year. While the strategy executes on daily close prices, we utilize a 3-year sliding window to ensure that our statistical tests, particularly the Engle-Granger cointegration, have a sufficient sample size (approximately 756 observations) to achieve statistical significance. For example, our training and validation sets utilized data from 2010 to 2012, while the 2013 data was reserved exclusively for out-of-sample testing. The preprocessing pipeline included the following:

- 3.1.1 Cleaning and Imputation: Each stock file was standardized into a consistent long-format schema with columns for date, prices, volume, and ticker, with dates parsed into proper timestamps and numeric fields coerced to valid types. Duplicate rows by date and ticker were removed, missing or invalid price entries were forward-filled within each stock’s history, and rows with missing dates or non-positive prices were dropped. To handle outliers, extreme daily returns occurring on abnormally low trading volume were flagged as suspicious, their closing prices masked and forward-filled, while genuine extreme moves with normal or high volume were retained as signals.
- 3.1.2 Feature Computation: We calculated log prices, $x_{i,t} = \ln(P_{i,t})$ where $P_{i,t}$ is the adjusted closing price of stock i at time t , and forward simple returns, $r_{i,t} = \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}}$.

3.2 Exploratory Data Analysis

3.2.1 Timeline Validation and Regime Detection

Before committing to a study window, we profiled data coverage and volatility regimes to confirm the backtest was grounded in a stable, representative sample. A coverage heatmap identified 2010 as the start date where 80% of the universe had consistent data, defining our 2010-2017 study period. We also detected major cross-sectional volatility spikes (up to 4x baseline) in 2010 and 2011. These identified crisis regimes informed the structuring of our training and validation windows to ensure model robustness.

3.2.2 Statistical Profiling and Outlier Investigation

On the filtered 2010-2017 dataset, we tested distribution and stationarity to verify the assumptions required for cointegration-based modelling downstream. The Augmented Dickey-Fuller (ADF) test confirmed prices were non-stationary $I(1)$ while log-returns were stationary $I(0)$, a prerequisite for cointegration. The return distributions showed extreme skewness and kurtosis. Moreover, we also measured the autocorrelation, where the return ACF was near zero for all 20 lags, confirming that signal extraction must rely on price-level relationships rather than return trends.

3.2.3 Downstream Signal Potential

The final stage of EDA quantified the search space to confirm that enough structurally related pairs exist to justify the clustering-then-cointegration pipeline rather than exhaustive pairwise testing. We analyzed the correlation distribution of $\approx 474,000$ pairs. While the universe centered around $r \approx 0.25$, we identified 925 pairs exceeding $r \geq 0.7$, providing a dense pool of candidates for the subsequent cointegration and clustering stages.

3.3 Pair Discovery and Statistical Testing

3.3.1 Principal Component Analysis (PCA)

We applied PCA to the forward simple return matrix to group stocks by common co-movement patterns. PCA was run separately over five fixed training sliding windows (2010–2012, 2011–2013, up to 2014–2016). For each window, we first constructed the return matrix ($N_{dates} \times N_{tickers}$) and standardized it to unit variance using `StandardScaler`. Since PCA requires samples as rows, the matrix was transposed so that each stock was treated as a sample and each trading day as a feature. We initially fit PCA with all components to obtain the full variance curve, then selected the smallest number of components n that captured at least 80% of cumulative variance. To avoid the curse of dimensionality during clustering, we imposed a cap such that $2 \leq n_{components} \leq 15$. Finally, PCA was refit with the chosen n components to obtain reduced coordinates for each stock.

3.3.2 Unsupervised Clustering

Using the reduced PCA coordinates as features, we evaluated four distinct unsupervised clustering methods to group stocks by co-movement patterns. This stage was critical for narrowing the search space from $O(N^2)$ potential pairs to localized groups of high-correlation candidates.

1. K-Means: This served as our initial baseline. It was eventually discarded because it requires a pre-defined cluster count (k) and assumes spherical cluster shapes, which failed to capture the non-linear manifold of stock return distributions.
2. Hierarchical Clustering: While useful for visualizing relationships via dendrograms, this method was computationally expensive and tended to produce overly broad clusters that lacked the statistical specificity required for downstream cointegration.
3. DBSCAN: We implemented this to find clusters of arbitrary shape. However, DBSCAN proved highly sensitive to the *epsilon* (ϵ) parameter; as the market density shifted across the fixed sliding windows, the algorithm often failed to find distinct groups, resulting in cluster depletion where many stocks were either merged into a few clusters or classified as noise.
4. OPTICS: Selected as our primary algorithm. Unlike the previous methods, OPTICS does not require a global density threshold and creates a reachability plot to identify clusters of varying densities. OPTICS consistently provided a granular, large, and stable number of clusters.

3.3.3 Cointegration and Pair Selection

Within each cluster identified by OPTICS, we executed a parallelized screening pipeline to identify pairs with strong mean-reverting properties. A pair was classified as eligible for trading only if it satisfied four distinct statistical criteria [3]:

1. Engle-Granger Cointegration Test: We performed the ADF test on the ordinary least squares (OLS) residuals of the log-closing price relationship. Pairs were retained only if the null hypothesis of a unit root was rejected ($p < 0.05$), ensuring a long-term equilibrium.
2. Mean Reversion Speed (Half-Life): To ensure the spread, defined as $S_t = Y_t - \beta X_t$ for pairs (x, y) , reverts to the mean within a tradable timeframe, we calculated the half-life ($t_{1/2}$) of the spread using

the approximation $t_{1/2} = \frac{\ln(2)}{|\beta|}$, where β is the rate of mean reversion estimated via OLS on the spread's first differences $\Delta S_t = -\beta(S_{t-1} - \mu)\Delta t + \epsilon_t$ for some noise ϵ_t and the spread's mean μ . We filtered for $1 \leq t_{1/2} \leq 252$ days to avoid pairs that take years to revert.

3. Hurst Exponent (H): To distinguish between mean-reverting and diffusive (random walk) behavior, we calculated the Hurst exponent. If $H < 0.5$, the behavior is mean-reverting. We are only interested in $H < 0.5$. If $H = 0.5$, it is a geometric Brownian motion, i.e., a random walk. Otherwise, if $H > 0.5$, the behavior is trending or diffusive.
4. Minimum Mean Crossings: To ensure the spread is frequently tradable, we enforced a threshold of at least 12 mean crossings per year. This prevents selecting pairs that are cointegrated but remain on one side of the mean for the entire training window.

To prioritize the highest quality pairs for the final modeling stage, we implemented a custom scoring heuristic. The score balances statistical significance with trading efficiency $Score = 0.60 \times (1 - p_{coint}) + 0.40 \times \left(1 - \frac{|t_{1/2} - 20|}{40}\right)$. This formula prioritizes pairs with a p -value near zero and a half-life close to a sweet spot of 20 days, i.e., monthly capital rotation. Pairs were then ranked per sliding window, and the top- K eligible candidates were passed to the feature engineering and machine learning pipeline. In our implementation, we set $K = 20$. This value was chosen to balance diversification and computational tractability; using too few pairs would make the portfolio overly concentrated and sensitive to individual pair failures, whereas using too many pairs would dilute statistical quality and significantly increase downstream training and backtesting cost.

3.3.4 Dynamic Hedge Ratio via Kalman Filter

For the pairs passing cointegration, we replaced the static OLS β with a 2D Kalman filter. While simple OLS provides a single snapshot of the relationship, the Kalman filter treats the hedge ratio as a hidden state that evolves over time, allowing the model to adapt to β -drift as market correlations shift. We modeled the relationship between Stock A (y_t) and Stock B (x_t) as a dynamic linear system:

- State Equation: The state vector $\theta_t = [\beta_t, \alpha_t]^T$, comprising the slope and intercept, is assumed to follow a random walk $\theta_t = \theta_{t-1} + w_t$ with $w_t \sim N(0, \delta I)$, where I is the 2×2 identity matrix and δ is the process noise variance (set to $1e^{-4}$), controlling the adaptation speed of the filter.
- Observation Equation: The observed price of Stock A is a linear function of Stock B plus the time-varying intercept $y_t = [x_t, 1] \begin{bmatrix} \beta_t \\ \alpha_t \end{bmatrix} + v_t$, with $v_t \sim N(0, R)$, where R is the observation noise variance determined from the parameter `obs_noise`.

More on the implementation, to ensure filter stability, we initialized the state vector θ_0 and the covariance matrix P using an OLS regression on the first 20 observations of the window to prevent initial noisy estimates from distorting the early spread. At each time step t , the filter calculates the difference between the actual and predicted price of Stock A and updates the hedge ratio using Kalman gain K .

3.4 Feature Engineering and Dataset Construction

3.4.1 Spread Feature Engineering

For each selected pair, we constructed both a static OLS-based spread and a dynamic Kalman-filtered spread. From these signals, we engineered the following feature categories:

- Mean Reversion Indicators: We computed rolling Z-scores of the spread across multiple horizons (e.g., 5, 21, and 63 days) to capture short-term and medium-term deviations from the equilibrium. This allows the model to identify if a spread is stretched relative to both its weekly and quarterly averages.
- Volatility and Risk: Rolling standard deviations and Parkinson volatility estimates were used to quantify the risk of spread widening and to normalize signals across different market regimes.
- Momentum and Trend: To detect whether a spread is actively reverting or trending away, we included the Rate of Change (ROC), Relative Strength Index (RSI), and moving average crossovers of the spread itself.
- Dynamic Derivatives: Using the Kalman filter state or output, we derived the rate of change of the hedge ratio $\Delta\beta_t$ to provide the model with information regarding the stability of the underlying stock relationship. A high $\Delta\beta_t$ suggests that the fundamental relationship between the stocks is shifting rapidly, signaling a potential structural break where mean reversion is less likely to occur.

3.4.2 Target Labeling

We implemented a labeling schema to cater to both classification and regression objectives. Unlike standard directional labeling, our approach focuses on the mean-reverting success of the spread:

- Binary Convergence Label: For discriminative classifiers such as XGBoost, we define a binary target $L_{t,h} \in \{0, 1\}$. A sample is labeled 1 if the absolute deviation from the mean decreases over the horizon h , and 0 otherwise: $L_{t,h} = \mathbb{I}(|S_{t+h} - \mu_{t+h}| < |S_t - \mu_t|)$, where μ is the 60-day (approximately 3 trading months) rolling mean, following the target half-time of 20 days. It is important to note that S_{t+h} is not calculated using β_{t+h} , but β_t instead, to isolate the model solely to predict the spread, not the

Kalman values. In other words, β is kept constant during the prediction but varies during training or other calculations; for future reference, say this as β -locked. This 60-day anchor provides a stable equilibrium baseline, ensuring the model identifies true structural reversion rather than chasing short-term price noise. By labeling convergence success rather than simple price direction, the classifier learns to estimate the statistical probability of a pair returning to its historical mean within the specified horizon.

- **Continuous Return Label:** For regression-based analysis and the LSTM sequence model, we calculate the raw h -day forward change in the spread $y_{t,h} = S_{t+h} - S_t$. This continuous target enables the model to estimate the magnitude of the expected move, which is essential for sizing positions and managing risk.

To ensure the integrity of the system and prevent look-ahead bias, these labels are constructed using a negative shift (e.g., `shift(-h)`). This strictly separates the future outcomes used for training from the present features used for prediction, ensuring that at any timestamp t in the backtest, the model only has access to information known at that moment.

3.5 Spread Modelling and Signal Prediction

The modeling phase tests six architectures chosen to probe different aspects of mean-reversion predictability: OU as the classical parametric benchmark, ARMA for linear temporal structure, Linear Regression as a linear-ML baseline, XGBoost for non-linear feature interactions, and LSTM/Encoder-Decoder for learned temporal representations. The ML tier ingests the full engineered feature set; OU and ARMA operate on the spread series alone.

3.5.1 Ornstein-Uhlenbeck (OU) Stochastic Baseline

The OU model serves as the fundamental statistical benchmark, treating the asset spread as a continuous-time mean-reverting stochastic process. The spread S_t is assumed to follow the Stochastic Differential Equation (SDE) [1] [4]: $dS_t = \kappa(\theta - S_t)dt + \sigma dW_t$, where κ is the rate of mean reversion, θ is the long-term equilibrium level, and σ is the diffusion coefficient. While the theory is continuous, our implementation utilizes a discretized AR(1) framework to estimate parameters via a walk-forward orchestrator. Since market data is sampled daily ($dt = 1$), we discretize the SDE into an Autoregressive process of order 1: $S_t = a + bS_{t-1} + \epsilon_t$, where $\epsilon_t \sim N(0, \sigma_\epsilon^2)$. We calculate the steady-state standard deviation $\sigma_{eq} = \frac{\sigma}{\sqrt{2\kappa}}$ to normalize the spread into a robust Z-score. This Z-score serves as the prediction of the trading signal. We created a walk-forward orchestrator to execute a sliding-window fit with lookback period of 60 days. At each timestep t , the model fits the previous 60 days of the spread to generate a point-in-time parameter set, which is then used to score the current observation. The model generates three distinct values for each pair at every timestamp:

1. Current Z-score: Defined as $z_t = \frac{S_t - \theta}{\sigma_{eq}}$, providing the current statistical deviation.
2. Expected spread: Calculated as $E[S_{t+h}] = S_t \cdot b^h + \theta(1 - b^h)$ for a $h = 10$ -day horizon.
3. Predicted change: The expected magnitude of the mean-reversion move over the next h days.

3.5.2 ARMA Stochastic Baseline

The Autoregressive Moving Average (ARMA) model serves as a linear time-series benchmark designed to capture the serial correlation and memory of the asset spread. While the OU process models the spread as a continuous mean-reverting force, the ARMA approach focuses on the discrete relationship between a spread's past values and its past shocks. We implement the ARMA(p, q) process, where p (Autoregressive) represents the number of lagged observations of the spread, and q (Moving Average) represents the number of lagged forecast errors. The model is defined as $S_t = c + \sum_{i=1}^p \phi_i S_{t-i} + \sum_{j=1}^q \theta_j \epsilon_{t-j} + \epsilon_t$, where ϵ_t is white noise and ϕ_i and θ_j are model parameters [3]. This structure allows the model to distinguish between persistent dependence in the spread (AR component) and transient market shocks (MA component). In our implementation, we use the open-source `statsmodels.tsa.arima` interface with differencing fixed at zero, i.e. `ARIMA(order=(p,0,q))`, so the main modeling decision is the selection of the lag orders (p, q).

We employ a global grid-search procedure implemented in `arma_tuning`. For each spread type, we evaluate candidate (p, q) values on the validation splits and rank them according to out-of-sample forecasting performance. The search is conducted over an 11×11 grid, where both the autoregressive order p and moving-average order q are varied from 0 to 10. Rather than selecting a separate order for every pair, we choose a single global (p, q) specification for each spread type and freeze it for the final holdout evaluation. This provides a consistent and scalable benchmark for comparison with the other predictive models. We select (p, q) using out-of-sample validation Directional Weighted Mean Squared Error (DWMSE), since our primary objective is predictive performance on future spread movement, with additional emphasis on penalizing wrong-direction forecasts rather than purely optimizing in-sample fit. We found that the optimal order is $(p, q) = (7, 2)$.

Given a fitted ARMA(p, q) model, the model's outputs are similar to what are being outputted by the OU baseline, except that the ARMA model directly predicts the future spread level $\hat{S}_{t,h}$. Finally, the

`arma_holdout_eval` module ensures that the selected global (p, q) parameters are held fixed and applied to the holdout period, allowing us to assess the stability of these linear relationships out of sample and to provide a benchmark against the more flexible non-linear models.

3.5.3 Linear Regression (Linear Benchmark)

As a secondary baseline for the machine learning tier, we implement a Linear Regression model to provide a linear reference for the spread prediction task. The model fits a linear combination of these features to predict the continuous h -day forward change in the spread: $\Delta\hat{S}_{t,h} = \beta_0 + \sum_{i=1}^n \beta_i x_i + \epsilon$, where $\Delta\hat{S}_{t,h}$ represents the predicted change in the spread level over the horizon h . We utilize the `sklearn.linear_model` framework, specifically employing Ridge Regression (L_2 regularization). This regularization is essential to handle multicollinearity among the engineered features, such as overlapping rolling Z-scores, by penalizing the squared magnitude of the coefficients β_i . The model is trained within a walk-forward framework, optimizing coefficients on the training partition and evaluating the Directional Weighted Mean Squared Error (DWMSE) on the validation set, defined as: $L_{DWMSE}(y, \hat{y}) = \frac{1}{2} (1 + \gamma \cdot \mathbb{I}(\text{sgn}(y) \neq \text{sgn}(\hat{y}))) (y - \hat{y})^2$, where $\gamma = 1.0$ is the direction penalty hyperparameter. The primary output is a continuous predicted change, which is then normalized by the current spread volatility to generate a model-adjusted Z-score for trade execution.

3.5.4 XGBoost

The XGBoost framework is implemented as a modular non-linear regression system that moves beyond simple binary classification by forecasting the expected price correction over a fixed 10-day horizon ($h = 10$). The architecture is centered around the `SpreadChangeXGBoost` class, which utilizes a gradient-boosted ensemble of decision trees from the `xgboost` module to optimize the DWMSE objective function. This allows the model to map the feature space to a continuous spread-change prediction. By capturing non-linear feature interactions, such as the relationship between volatility regimes and mean-reversion speed, the model provides a more granular assessment of convergence than traditional linear or threshold-based approaches. To ensure the model generalizes across diverse market regimes, it undergoes a rigorous hyperparameter optimization process. Parameters such as tree depth (`max_depth`), ensemble size (`n_estimators`), and the learning rate (η) are tuned using a grid search that evaluates performance based on the DWMSE across multiple sliding validation folds, ensuring that the chosen configuration remains robust against non-stationary data and shifting volatility environments. In our experiments, the best-performing configuration was `max_depth=3`, `n_estimators=100`, and `learning_rate=0.01`. This structured optimization allows the ensemble to effectively weight individual decision trees to minimize residual errors in the predicted spread trajectory.

The final output of the model is a continuous scalar representing the predicted spread change $\Delta\hat{S}_{t+h}$. Crucially, this prediction is β -locked at the forecast origin t , assuming a constant hedge ratio for the duration of the trade. This raw forecast is then transformed into two actionable metrics: a predicted spread value $s_t + \Delta\hat{S}_{t+h}$, representing the expected price target, and a predicted Z-score $\frac{\Delta\hat{S}_{t+h}}{\sigma_{20d}}$. By normalizing the prediction by the rolling volatility at the origin, the system generates a standardized conviction signal, enabling uniform entry and exit thresholds across the entire universe of cointegrated asset pairs.

3.5.5 LSTM and Encoder-Decoder Sequence Models

To capture the temporal memory of price action, we implement LSTM networks and Encoder-Decoder (Seq2Seq) architectures. Unlike snapshot-based models, these process the historical trajectory of the spread leading up to the forecast origin t . Input sequences of shape $(\text{batch}, w, 11)$ are constructed using a sliding lookback window w , allowing the network to identify signatures of spread behavior, such as distinguishing between structural drift and mean-reverting spikes. During backpropagation through time (BPTT), the gradients are calculated by unrolling the network, allowing the model to learn the long-term dependencies of the cointegration relationship. The final hidden state is mapped to a continuous output representing the 10-day spread change ($\Delta\hat{S}_{t+h}$). In our final specification, the LSTM model uses the Kalman spread variant with hidden size 64, window size 10, and learning rate 0.001.

The Encoder-Decoder variant extends this by separating historical processing from future projection. The encoder processes the w -day history to produce a final hidden state that serves as the context vector. This vector is passed to the decoder, which is initialized with the encoder's state. In a sequence-to-sequence fashion, the decoder iteratively generates a path of expected spread changes. For training, we utilize teacher forcing, where the ground truth from the β -locked labels is fed back into the decoder at each step to stabilize learning. At inference, the decoder's own previous prediction is used as the next input. The final predicted change is added to the spread value at the origin. In our final specification, the Encoder-Decoder model also uses the Kalman spread variant, with hidden size 64, window size 20, and learning rate 0.0005.

3.6 Signal Standardization and Decision Logic

To facilitate a fair comparison across independent architectures, we implement a standardization layer that maps heterogeneous model outputs, ranging from OU mean-reversion speeds to LSTM price-path forecasts, into a common decision space. Each model’s raw prediction is converted into a model-adjusted Z-score Z_{adj} . For the stochastic models, this is derived from the steady-state deviation of the spread from its estimated equilibrium. For the machine learning models, Z_{adj} is calculated by normalizing the 10-day predicted change ($\Delta\hat{s}_{t,h}$) by the rolling 20-day Parkinson volatility (σ_{20d}) of the spread at the forecast origin. A trade is only executed if $|Z_{adj}|$ exceeds a specific confidence threshold (e.g., $Z_{adj} > 2.0$ for shorts or $Z_{adj} < -2.0$ for longs). By utilizing this uniform thresholding logic, the final execution engine remains identical across all architectures. This allows us to isolate the predictive accuracy of the model itself as the primary variable influencing the strategy’s Sharpe ratio, which measures risk-adjusted performance, and Alpha, which represents the excess return of a strategy relative to a benchmark.

3.7 Trading Strategy and Backtesting

The backtesting framework is engineered to simulate a high-fidelity institutional trading environment, transforming the standardized model signals into a series of discrete market operations.

3.7.1 Signal Generation and Quantile-Based Optimization: While baseline entry thresholds are conceptually set at ± 2.0 , the `quantile_sweep` module performs a data-driven refinement of these levels to account for the unique distribution of each spread. This module executes an exhaustive grid search over a range of quantiles (e.g., 5th to 20th percentiles for long entries and 80th to 95th for short entries) within the training and validation partitions. For each candidate quantile, the system evaluates key performance indicators such as the Profit Factor, Sharpe Ratio, and Maximum Drawdown. The resulting `QuantileZScoreSignal` dynamically maps these optimized thresholds to the current spread distribution, ensuring the entry logic adapts to the specific volatility and tail-risk characteristics of each pair rather than relying on a static Gaussian assumption.

3.7.2 Vectorized Backtesting Engine and State Logic: The core execution logic, `backtest_engine`, utilizes a high-performance vectorized approach to simulate trading across the entire historical timeline. The engine ingests the output from `forecast_signal`, which converts raw spread-change forecasts ($\Delta\hat{s}_{t+h}$) into standardized Z_{adj} signals, to track state transitions between "Long Spread," "Short Spread," and "Neutral" positions. The engine enforces a rigorous path-dependent state machine:

- **Entry Condition:** A position is opened when $|Z_{adj}| > \delta_{entry}$, where δ_{entry} is the model-specific optimized threshold.
- **Exit Condition:** To capture the full mean-reversion move while minimizing exposure time, the engine utilizes a dual-exit logic. A trade is liquidated when either $|Z_{adj}|$ falls below a soft threshold δ_{exit} (typically 0.5) or when the signal undergoes a sign flip ($Z_t \cdot Z_{t-1} < 0$), indicating the spread has fully crossed its equilibrium point.
- **Persistence:** The logic prevents signal flickering by locking the active state until a valid exit is triggered, ensuring realistic holding periods and reducing unnecessary turnover.

3.7.3 Friction and Transaction Cost Modeling: To ensure that the reported Alpha is not an artifact of frictionless simulation, the engine incorporates a structural cost layer. Every transaction, both entry and exit, is penalized with a 10 basis point (0.1%) cost per leg. This total friction covers brokerage commissions, regulatory fees, and the bid-ask slippage inherent in the execution of mid-cap equity pairs. This hurdle rate effectively filters out marginal signals that appear profitable in a vacuum but fail to generate net risk-adjusted returns in a competitive market environment.

3.7.4 Walk-Forward Evaluation and Temporal Robustness: The strategy is executed through a five-window walk-forward orchestrator to prevent look-ahead bias and address the non-stationarity of the financial time series. As defined in the backtesting pipeline, each window is partitioned into three distinct phases: a) **Training Phase:** Estimation of model parameters (e.g., XGBoost weights or LSTM hidden states), b) **Validation Phase:** Execution of the `quantile_sweep` to identify the optimal entry/exit thresholds, and c) **Test Phase:** A non-overlapping out-of-sample execution where the "locked" model and thresholds are applied to unseen data. This rolling approach ensures that the strategy’s parameters, including the 60-day lookback for Kalman filtering and the 10-day forecast horizon, remain robust as the underlying cointegration relationships evolve across different market regimes.

3.7.5 Dynamic Portfolio Selection: At each daily timestep, the system evaluates the entire universe of cointegrated pairs through a portfolio selection mechanism. Capital is dynamically allocated to the top N pairs displaying the highest signal conviction (absolute Z_{adj}). This layer includes structural risk controls: if a pair’s cointegration strength degrades, detected via a spike in the Kalman β change variance or a failure in the recursive ADF test, the engine triggers an emergency liquidation. This ensures that the portfolio is only exposed to high-probability mean-reversion events where the statistical relationship remains fundamentally intact.

3.8 Robustness and Final Evaluation

To ensure that the performance of the non-linear models is not a localized artifact of specific parameter configurations, we subject the strategy to a multi-layered robustness framework. This phase validates the stability of the predictive Alpha across varying market conditions and internal model settings.

- 3.8.1 **Hyperparameter Sensitivity Analysis:** We conduct an extensive sensitivity sweep over the strategy’s core parameters. This involves perturbing the rolling z-score lookback window, the number of traded pairs, transaction costs, and the entry/exit quantile thresholds ($\delta_{entry}, \delta_{exit}$). A robust strategy must exhibit a smooth performance profile; sharp drops in the Sharpe Ratio following minor parameter adjustments would indicate a brittle over-fit. We specifically analyze the sensitivity of the **quantile_sweep** results to ensure that the optimal entry and exit thresholds are not outliers within their local parameter space, and validate that switching from fixed z-score rules to adaptive quantile thresholds is a genuine and robust improvement.
- 3.8.2 **Hedge Ratio Stability Testing:** Since the spread is fundamentally dependent on the estimation of β , we evaluate the robustness of our signals against different hedge ratio methodologies. This includes a comparative analysis between the dynamic Kalman Filter approach and a rolling OLS baseline. By measuring the variance of the Kalman β updates, we quantify the structural stability of the pairs. Pairs exhibiting excessive β volatility are flagged as regime-unstable and are excluded from the final evaluation to prevent capital erosion during structural breaks.
- 3.8.3 **Aggregated Walk-Forward Performance:** The ultimate measure of the strategy’s efficacy is the aggregated performance across all five non-overlapping walk-forward windows. Rather than cherry-picking the best performing period, we calculate the global Sharpe ratio and maximum drawdown by concatenating the out-of-sample (OOS) equity curves. This provides a statistically significant representation of the strategy’s performance across diverse volatility regimes, from low-volatility drift to high-volatility market shocks.
- 3.8.4 **Unbiased Holdout Evaluation:** As a final safeguard against data snooping, a single, held-out dataset, representing the most recent market period, is reserved for the absolute final execution. This holdout test is performed only once, using the frozen parameters and weights derived from the previous walk-forward cycles. The results from this holdout period serve as the primary unbiased estimate of the strategy’s expected performance in a live production environment.

4 Results and Discussion

4.1 Model Comparison

We compared all major Kalman-based model families on the 2017 holdout set using Directional Weighted Mean Squared Error (DWMSE) as the primary metric, with directional accuracy reported as a complementary measure. Among all evaluated models, ARMA achieved the best overall performance, recording the lowest DWMSE of 0.000897 and the highest directional accuracy of 58.1%. This indicates that the ARMA specification provided the best balance between forecast magnitude and sign correctness on the final holdout period. XGBoost ranked second by DWMSE at 0.000909, very close to ARMA, but its directional accuracy was only 50.3%, suggesting that its error performance was driven more by controlling forecast magnitude than by consistently predicting the correct direction of spread movement.

The sequence-based models produced weaker results overall. The encoder-decoder model achieved a DWMSE of 0.001081 with directional accuracy of 52.4%, outperforming the vanilla LSTM of a DWMSE of 0.001296 and directional accuracy of 50.5%. The OU benchmark obtained a DWMSE of 0.001800 and directional accuracy of 49.3%, placing it below the stronger supervised models but above linear regression. Linear regression performed worst overall, with a DWMSE of 0.002007 and directional accuracy of only 46.5%, indicating that a simple linear mapping from engineered features to spread change was insufficient for robust holdout forecasting. Overall, these results show that on the final 10-day spread forecasting task, the strongest performance came from models that could capture structured linear time-series dependence directly, with ARMA providing the most reliable OOS benchmark.

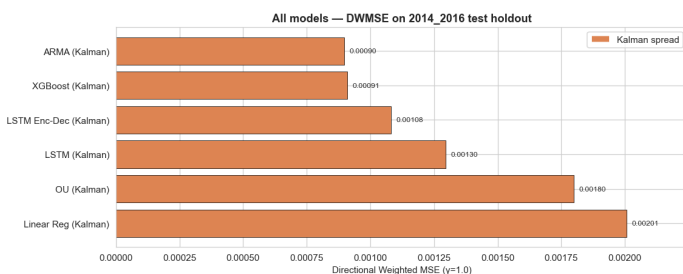


Figure 1: Model Comparison by DWMSE

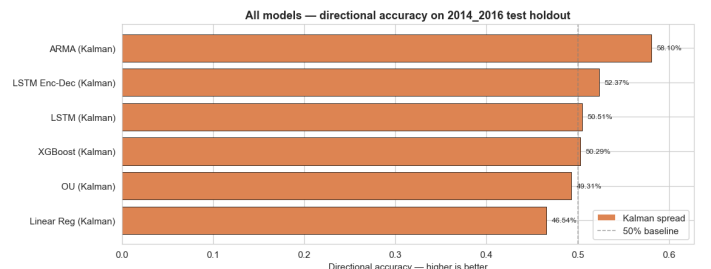


Figure 2: Model Comparison by Directional Accuracy

4.2 Sensitivity Analysis

Sensitivity analysis identified the stop-loss threshold, entry/exit rules, transaction costs, and lookback window as high-impact variables. Key findings from the optimization include the following:

- 4.2.1 Stop-Loss Impact: Implementing a stop-loss degrades performance by triggering excessive trading and exits before mean reversion occurs.
- 4.2.2 Adaptive Thresholds: Quantile-based entry/exit rules significantly outperform fixed Z-scores by adapting to local volatility regimes.
- 4.2.3 Cost Sensitivity: Profitability is marginal only under zero transaction costs; profit-per-trade currently fails to offset realistic market slippage.
- 4.2.4 Lookback Optimization: The optimal 126-day window provides the best balance between signal stability and responsiveness.

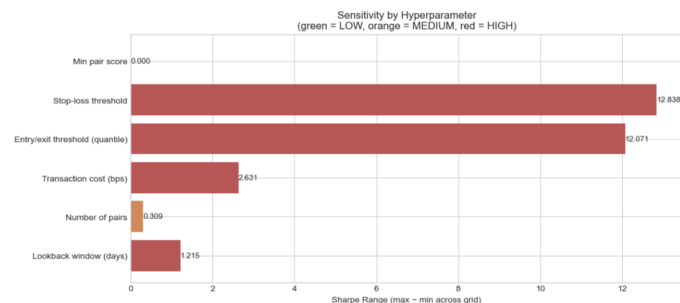


Figure 3: Hyperparameter Sensitivity Measured by Sharpe Range

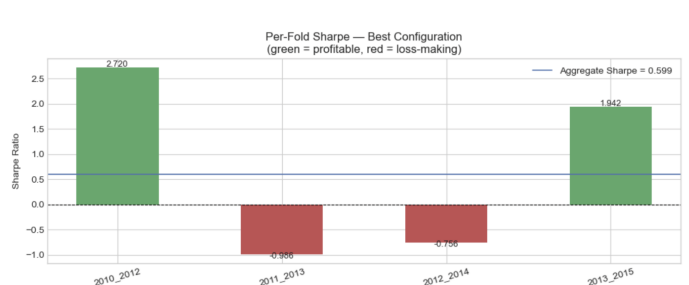


Figure 4: Sharpe Ratio of the Best Configuration per Window

The strategy's biggest problem is that it does not make enough profit per trade to cover the cost of making that trade. The best way to fix this is simply to trade less, which means using a longer lookback window and waiting for a bigger price gap before entering a trade. The best configuration achieves

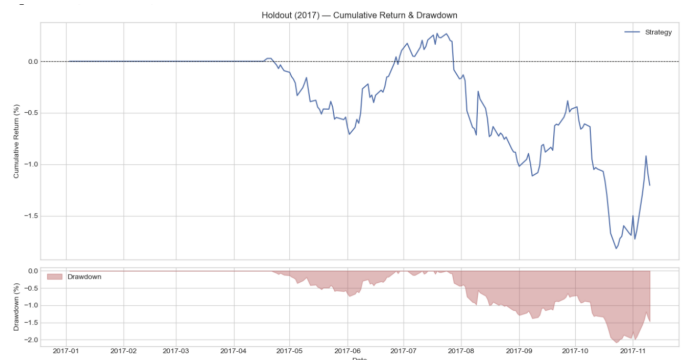


Figure 5: Cumulative Return and Drawdown of the Best Configuration Across All Validation Folds (2013-2016) and Holdout Period (2017)

an aggregate Sharpe of 0.599, but the per-fold breakdown reveals this is not consistent. The strategy is profitable in only 2 out of 4 folds, strongly in 2013 (Sharpe = 2.720) and 2016 (Sharpe = 1.942), but loss-making in 2014 (-0.986) and 2015 (-0.756). The positive aggregate is driven entirely by the two strong years, masking the losses in between. The cumulative return chart tells the same story: a steady climb to +3% in 2013, followed by a sharp decline through 2014–2015 with drawdowns reaching -3% to -4%, before a strong recovery to +5.3% by the end of 2016. The strategy is sensitive to market conditions, and its performance is not yet stable enough for reliable deployment.

Moreover, the holdout test runs the best configuration on 2017 data exactly once, giving an unbiased estimate of real-world performance. The strategy loses money: Sharpe = -1.01, annual return = -1.39%, with only 76 trades made all year. The losses are not from overtrading; the strategy was selective, but the trades it took mostly failed. The cumulative return chart shows the strategy sitting flat for the first four months with no opportunities, briefly turning profitable in July (+0.2%), then declining steadily through August to November to a low of -1.7% before partially recovering to -1.2% at year's end. Max drawdown reached -2.1% in late October with no recovery. 2017 was an exceptionally strong bull market, which is one of the worst environments for pairs trading, as stocks tend to trend together in one direction rather than mean-revert. The holdout result confirms that the strategy is not yet robust enough for consistent real-world deployment.

4.3 SHAP Analysis

SHapley Additive exPlanations (SHAP) values were computed across four model families to identify features driving mean-reversion predictions. All models utilized Kalman variants, which consistently outperformed their OLS counterparts by 9–15% on DWMSE.

- 4.3.1 Hedge Ratio Drift Dominates Tabular Models: In linear regression and XGBoost, the rate of change in the hedge ratio (`kalman.beta.change`) is the most critical feature, accounting for 39.4%

Model	MSE	MAE	Dir. Acc.
Linear Regression	0.000759	0.018691	64.5%
XGBoost	0.000777	0.018638	63.6%
Encoder-Decoder	0.001129	0.023943	60.1%
LSTM	0.001418	0.027131	58.3%

Figure 6: Model Performance Summary after Backtesting (2017 Holdout)

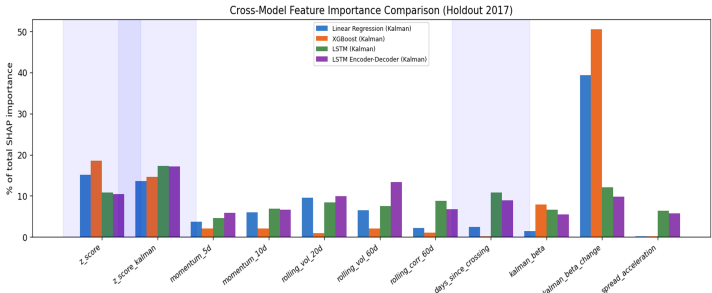


Figure 7: Cross-Model Feature Importance Comparison

- and 50.5% of total importance, respectively. This suggests that relationship stability is the primary gatekeeper for tradeability in tabular models.
- 4.3.2 Mean-Reversion Confirmation: XGBoost’s Z-score dependence plot exhibits a clean antisymmetric pattern, where negative Z-scores produce positive SHAP values (predicting a rise back to zero) and vice versa, capturing the expected economic prior.
- 4.3.3 Sequential Model Shift: LSTM and Encoder-Decoder models shift priority to the Kalman’s Z-score ($\approx 17\%$). These architectures distribute importance more evenly across the 20-step temporal window, with the Encoder-Decoder also prioritizing long-term rolling volatility.
- 4.3.4 Minimal Overfitting: Arbitrary lag features like momentum and `days_since_crossing` consistently rank low, indicating the models did not overfit to lagged noise.

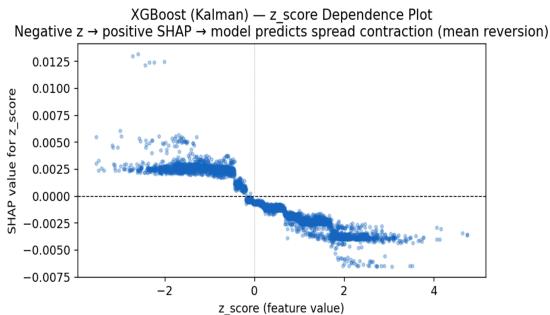


Figure 8: XGBoost Z-score Asymmetric SHAP Dependency Plot

Category	Dir. Acc.	MAE
Small moves	57.6%	—
Medium moves	67.3%	—
Large moves	65.7%	—
Correct predictions (avg error)	—	0.0141
Wrong predictions (avg error)	—	0.0267 (1.9× larger)

Figure 9: Accuracy-Profitability Gap

4.4 Microscopic Evaluation

XGBoost was selected as the representative model for microscopic analysis due to its superior interpretability and high directional accuracy of 63.6%. Despite this predictive edge, the strategy remains unprofitable in backtesting, necessitating a granular examination of successful versus failed trades.

- 4.4.1 The Accuracy-Profitability Gap: The primary impediment to profitability is a structural error asymmetry. As shown in Table 9, while the model is 65.7% accurate on large spread moves, the magnitude of error when the model is incorrect (0.0267) is nearly $1.9\times$ larger than the error of correct predictions (0.0141). Under the DWMSE $2\times$ penalty, a single wrong-direction trade contributes approximately $7\times$ more loss per observation than a correct trade contributes in gain, effectively erasing the statistical edge provided by the 63.6% accuracy.
- 4.4.2 Successful vs. Failed Trade Dynamics: Microscopic analysis of individual trade paths reveals a definitive link between hedge ratio stability and trade outcome.
- **Successful Trades:** Characterized by near-zero `kalman_beta_change` values ($\leq 1.2 \times 10^{-5}$). In these instances, the spread represents a genuine temporary divergence from a stable mean, allowing for clean reversion within 10-15 days.
 - **Failed Trades:** These observations exhibit `kalman_beta_change` values three to four orders of magnitude larger (0.003-0.009). High Z-score signals (up to ± 2.4) in these regimes are unreliable as they mask structural breaks and regime drift as mere mean-reversion opportunities.
- 4.4.3 Actionable Insight: The microscopic evaluation confirms that high Z-scores are insufficient for trade entry. To mitigate high-magnitude losses, the strategy requires a `kalman_beta_change` filter to restrict trading to regimes where the underlying pair relationship is structurally stable.

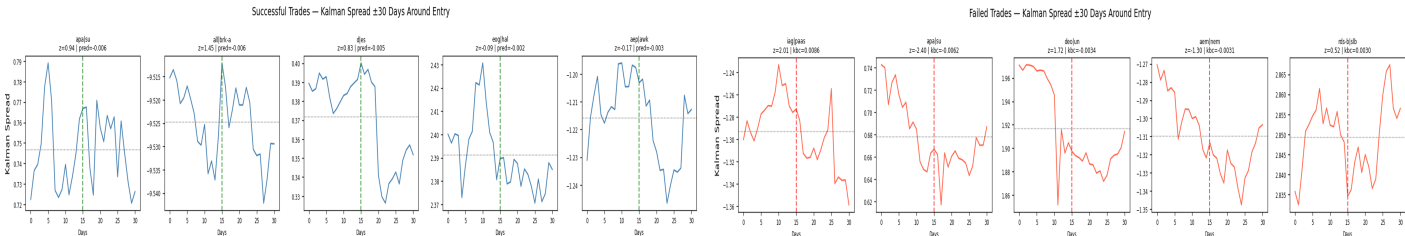


Figure 10: Success vs. Failed Trade Pair Dynamics

5 Conclusions

Our project addressed whether a machine learning pipeline can outperform static threshold-based pairs trading once realistic transaction costs and non-stationary market regimes are accounted for. The answer is partial. We found two main results.

First, ARMA is the strongest pure forecaster on the 2017 holdout (DWMSE 0.000897, 58.1% directional accuracy), while XGBoost is the most interpretable and actionable model through its tight coupling with SHAP-based feature diagnostics. The sequence models (LSTM, Encoder-Decoder) sat in the middle of the pack, suggesting that with our sample size the engineered-feature tabular models recover most of what a learned temporal representation could extract.

Second, and most importantly, all models exhibit an accuracy-profitability gap. The best configuration achieves an aggregate Sharpe of 0.599 across 2013–2016 but is highly regime-dependent (Sharpe = 2.72 in 2013 and 1.94 in 2016, against -0.99 in 2014 and -0.76 in 2015), and the 2017 holdout lost money at Sharpe = -1.01 . The microscopic error analysis in Section 4.4 localises the cause: wrong-direction trades produce an average error magnitude roughly $1.9\times$ larger than correct trades, and these wrong trades cluster in regimes where `kalman_beta_change` is three to four orders of magnitude above the stable-trade baseline. High-conviction Z-score signals in these regimes mask structural breaks as mean-reversion opportunities.

The project’s contribution is therefore twofold. It produces a measurable forecasting edge across multiple model families on the spread-prediction task, and it identifies, from within the data itself, the single most actionable next change: a `kalman_beta_change` regime filter that restricts trading to structurally stable pair relationships. The value lies less in the headline Sharpe number than in the diagnostic clarity about what must change to close the accuracy-profitability gap.

5.1 Limitations

Several constraints bound the generality of our findings. The evaluation is limited to US equities from 2010–2017 with 2017 as the final holdout, so the conclusions cannot be extrapolated to other markets, asset classes, or more recent periods without further testing. This matters for interpretation: our negative 2017 Sharpe coincides with a uniquely low-volatility trending bull market, and a different holdout year would likely have produced a different verdict on the pipeline’s viability. Relatedly, the top-1,000 universe is ranked on dollar volume computed over the full dataset, so stocks that delisted during the study window are systematically under-represented, likely inflating the tradeability of our cohort relative to a live system.

A second class of limitation concerns the framework itself. The DWMSE training objective is not aligned with trading P&L: it penalises sign and magnitude errors but does not capture the magnitude floor below which forecasts cannot clear friction, the timing of realisation, or regime validity, so a model can score well on DWMSE while still losing money in execution. Regime instability is also only partially handled: the microscopic analysis in Section 4.4 shows failed trades cluster when `kalman_beta_change` spikes, but the current pipeline uses this signal only as an input feature rather than as a trade-execution filter, leaving the strategy exposed to structural breaks. Finally, computational constraints capped the scope of hyperparameter and architecture search: ancillary hyperparameters such as the 60-day label mean and 20-day volatility normalisation are fixed throughout, and larger model architectures or finer grid searches were not feasible within the project’s compute budget. These limitations mean that reported performance should be read as a lower bound on what a more thoroughly tuned version of the same pipeline could achieve.

5.2 Future Directions

The most direct next step is to handle regime instability explicitly by implementing a `kalman_beta_change` regime filter at the execution layer. The microscopic analysis predicts this would remove the largest concentrated source of losing trades, since failed trades exhibit `kalman_beta_change` values three to four orders of magnitude above the successful-trade baseline; filtering these out directly attacks the asymmetric error mechanism identified in Section 4.4. A second promising direction is to perform pairs trading within grouped industries rather than across the full universe, since stocks in the same industry share exposure to common fundamental drivers that make their long-run relationships more economically anchored and less prone to structural breaks; this should raise the baseline cointegration quality of candidate pairs before any model-level improvements. Finally, moving from the current per-pair logic to portfolio-level optimisation would let the strategy allocate capital by jointly accounting for cross-pair correlations and total exposure, rather than treating each pair as independent; this addresses the concentration risk visible in the 2014–2015 folds, where sector-level regime shifts caused multiple pairs to fail together.

References

- [1] William K. Bertram. Analytic solutions for optimal statistical arbitrage trading. *Physica A: Statistical Mechanics and its Applications*, 389(11):2234–2243, 2010.
- [2] Boris Marjanovic. Huge stock market dataset. <https://www.kaggle.com/datasets/borismarjanovic/stock-market-dataset>, 2020. Accessed: 2026-04-17.
- [3] Simão Moraes Sarmento and Nuno Horta. *A Machine Learning Based Pairs Trading Investment Strategy*. Springer Briefs in Applied Sciences and Technology: Computational Intelligence. Springer, 2021. Accessed: 2026-04-17.
- [4] Ganapathy Vidyamurthy. *Pairs Trading: Quantitative Methods and Analysis*. Wiley Finance. John Wiley & Sons, 2004.

STATEMENT OF GROUP INDEPENDENT WORK

Declaration of Original Work

By entering our Student ID below, we certify that we completed our assignment independently of all except as stated below.

Signed,

1. A0305671H
2. A0276057X
3. A0305741L
4. A0303278E
5. A0284680W
6. A0305657Y

References

We give credit where credit is due. We acknowledge that we used the following websites, AI models, or contacts to complete this assignment:

- Claude Sonnet 4.6 and Claude Opus 4.7, which assisted in developing the models used in our project.
- Gemini 3 Flash and ChatGPT, which provided advice and insights on the results, and helped us verify our findings.